Fine-Tuning a Transformer Model for Multiple-Choice Question Answering

RoBERTa-base with LoRA on the Smart MCQ Solver Challenge (Kaggle)

Naini Diwan

July 2026

1. Introduction

This study fine-tunes a transformer encoder to solve five-option multiple-choice questions for the "Smart MCQ Solver Challenge" Kaggle competition, hosted by IIT Madras. The notebook cleans a 2,000-row training set, builds a PyTorch dataset that pairs each question with its five candidate answers, and fine-tunes a RoBERTa base model adapted for the AutoModelForMultipleChoice head using Low-Rank Adaptation (LoRA), a parameter-efficient fine-tuning (PEFT) technique.

Model quality is tracked with three metrics: top-1 accuracy, macro F1-score, and Mean Average Precision at 3 (MAP@3), the last of which matches the competition's scoring metric because submissions request the top three ranked answers per question. Training is logged to Weights & Biases (W&B), and the fine-tuned model is used to generate top-3 ranked predictions on the held-out test set.

The notebook is titled "Model-3: Fine-Tuned RoBERTa + LoRA Layers", which implies two earlier baseline models were built prior to this one. Those earlier stages are not present in the uploaded notebook, so this report covers only the RoBERTa + LoRA pipeline that is actually implemented.

Objective

The task is a single-label, multi-class classification problem: given a question prompt and five candidate answers (A through E), the model must identify the correct answer. The competition additionally scores submissions on MAP@3, so the pipeline is built to output a ranked list of the three most likely answers per question rather than a single prediction.

Dataset

Column	Description
id	Row index
prompt	Question text
A – E	Five candidate answer texts
answer	Ground-truth label (train.csv only), one of A–E

The training set contains 2,000 rows and 8 columns, with no missing values.

2. Data Preprocessing

2.1 Cleaning steps

- Null check: confirmed no missing values in prompt and answer-option columns.
- Duplicate removal: dropped 242 duplicate prompts, retaining the first occurrence per group.
- Case normalization: lower-cased all text in prompt and option columns for both train and test sets.
- Boilerplate stripping: a regular expression removes leading instructional phrases such as "pick the best possible answer:", "select the most accurate option:", etc.
- Label encoding: scikit-learn's LabelEncoder is fit on the answer column to produce numeric labels.

2.2 Train/validation split

The cleaned training data is split 80/20 into training and validation sets using train_test_split with a fixed random_state=42, stratified on the answer column. Stratification was chosen over a plain random split to keep the A–E answer distribution consistent across both sets and avoid class imbalance. The resulting split produced 1,406 training rows and 352 validation rows.

2.3 Reproducibility

A global seed (42) is applied to Python's random module, NumPy, and PyTorch (including CUDA), and cuDNN is set to deterministic mode with benchmarking disabled, so results are reproducible across runs on the same hardware.

3. Evaluation Metrics

A custom calculate_metrics function is passed to the Hugging Face Trainer and computes three metrics from the model's raw logits and true labels:

- Accuracy: top-1 exact match between the highest-probability prediction and the true label.
- Macro F1-score: F1 averaged evenly across the five answer classes, unweighted by class frequency.
- MAP@3: for each example, a score of *1/rank* is awarded if true label appears within the top three predictions (1.0 if ranked first, 0.5 if second, 0.33 if third, 0 otherwise), averaged over all examples.

np.argsort is used instead of a simple max/argmax specifically because MAP@3 needs the full top-3 ranking of predictions, not just the single best answer.

4. PyTorch Dataset and Data Collator

A custom HuggingFaceMCQDataset class (subclassing torch.utils.data.Dataset) formats each row for the multiple-choice model interface. For every question, it builds five (prompt, option) text pairs and tokenizes them together with padding to a fixed max_length and truncation enabled, producing input_ids and attention_mask tensors of shape [num_choices, max_length]. Fixed-length padding and truncation were chosen deliberately over dynamic-length batching so that every example forms a uniform rectangular tensor of shape [batch_size, num_choices, max_length]. During training, the encoded answer label is attached as a tensor but during inference this is omitted.

mcq_data_collator function stacks the per-example tensors from a list of dataset items into batched tensors using torch.stack, and conditionally stacks labels only when they are present, allowing the same collator to serve both training and test-time DataLoaders.

Pseudocode:

```
class HuggingFaceMCQDataset(Dataset):
    def __init__(self, df, tokenizer, max_len=256, is_test=False):
        ...
    def __getitem__(self, idx):
        # builds 5 (prompt, option) pairs per row
        # tokenizes with padding="max_length", truncation=True
        # attaches labels tensor if not is_test

def mcq_data_collator(features):
    # torch.stack over input_ids / attention_mask / labels
```

5. Model: RoBERTa-base + LoRA

The model's base is RoBERTA (Robustly Optimized BERT Approach), loaded via AutoModelForMultipleChoice architecture. It is adapted with a multiple-choice classification head that scores each of the five (question, option) pairs and outputs a single logit per option.

LoRA configuration

Rather than fine-tuning all of RoBERTa's parameters, we applied LoRA (Low-Rank Adaptation) through the PEFT library, which freezes the base model weights and injects small trainable rank-decomposition matrices into selected layers. This substantially reduces the number of trainable parameters.

Parameter	Value	Purpose
r	8	Rank of the LoRA update matrices
lora_alpha	16	Scaling factor for LoRA updates
target_modules	query, value	Attention sub-layers adapted
lora_dropout	0.1	Dropout applied within LoRA layers
bias	none	No bias parameters trained
task_type	SEQ_CLS	Sequence classification task type

Training configuration

Training is orchestrated with the Hugging Face Trainer and TrainingArguments, summarized below.

Parameter	Value
learning_rate	5e-4
per_device_train_batch_size	4
gradient_accumulation_steps	4 (effective batch size 16)
per_device_eval_batch_size	4
num_train_epochs	12
lr_scheduler_type	cosine
warmup_ratio	0.03 (~6% of total steps)
eval_strategy / save_strategy	epoch / epoch
load_best_model_at_end	True
metric_for_best_model	eval_map@3 (greater is better)
logging / tracking	logging_steps=10, report_to="wandb"

Evaluation and checkpoint saving both run on an epoch cadence so that load_best_model_at_end can restore the checkpoint with the strongest validation MAP@3 rather than simply the last epoch trained. Metrics are logged to Weights & Biases under the run name *peft-roberta-lora-run*.

Epoch	Training Loss	Validation Loss	Accuracy	F1 Score	Map@3
1	12.848639	3.214296	0.355114	0.347489	0.509943
2	11.976170	2.697126	0.428977	0.421961	0.614110
3	9.330994	2.174770	0.610795	0.606338	0.727746
4	7.723852	1.723459	0.701705	0.701520	0.778883
5	6.657336	1.424450	0.784091	0.785690	0.828598
6	5.678540	1.266002	0.803977	0.805444	0.852746
7	4.416777	1.051150	0.818182	0.818375	0.869318
8	4.896526	1.093020	0.832386	0.833473	0.876420
9	4.744751	0.954756	0.835227	0.835922	0.885417
10	4.543269	0.922807	0.840909	0.841412	0.891572
11	3.940620	0.910892	0.840909	0.842103	0.892992
12	4.053871	0.908904	0.840909	0.842171	0.892992

6. Results

For test set, the same HuggingFaceMCQDataset is wrapped in DataLoader, with shuffle disabled to preserve row order. The fine-tuned PEFT model is set to evaluation mode, and predictions are generated under torch.no_grad() to avoid unnecessary gradient computation and memory use during inference.

Raw logits from the model are converted to probabilities with a softmax over the five options. While, sigmoid outputs are intended for multi-label settings where several classes can be simultaneously true.

For each test question, three options with the highest predicted probability are selected and mapped back from numeric indices to their A–E letters. Then they are joined into a space-separated string (for example, “B D A”), matching the top-3 ranked format the competition's MAP@3 scoring expects.

Evaluation results over the validation data:

map@3	0.89299
accuracy	0.84091
f1 score	0.84217
loss	0.9089

7. Key Decisions

Decision	Rationale
Stratified train/validation split	Preserves A–E answer proportions in both sets, avoiding class imbalance.
Fixed-length padding/truncation	Produces uniform [batch, num_choices, max_length] tensors required by the multiple-choice model and GPU batching.
argsort over argmax for metrics	MAP@3 needs the full top-3 ranking, not just the single best prediction.
LoRA instead of full fine-tuning	Parameter-efficient adaptation of RoBERTa via small trainable rank-decomposition matrices.
eval_map@3: model-selection metric	Aligns checkpoint selection with the competition's actual scoring metric.
Softmax over sigmoid at inference	Task is single-label, multi-class (1 of 5 options is correct), not multi-label.